

TMDB Movie Pipeline

End-to-End Data Pipeline: Bronze → Silver → Gold → Dashboard

Tools & Technologies

PostgreSQL · dbt · PySpark · Apache NiFi · Apache Kafka · Debezium ClickHouse · MinIO ·
Streamlit · Docker · Python

By: Nabila Hulwana Z.

Mini Bootcamp Data Engineering rubythalib.ai

Instructor: Kak Guntur

Periode : February – April 2026

Project GitHub: <https://github.com/nabilahlw/tmdb-pipeline-project.git>

Project Overview

This project builds an end-to-end data pipeline for the TMDb (The Movie Database) movie dataset using Medallion Architecture (Bronze → Silver → Gold). Data is sourced from two places: a Kaggle dataset (4,803 films) and the TMDb live API.

Data Sources:

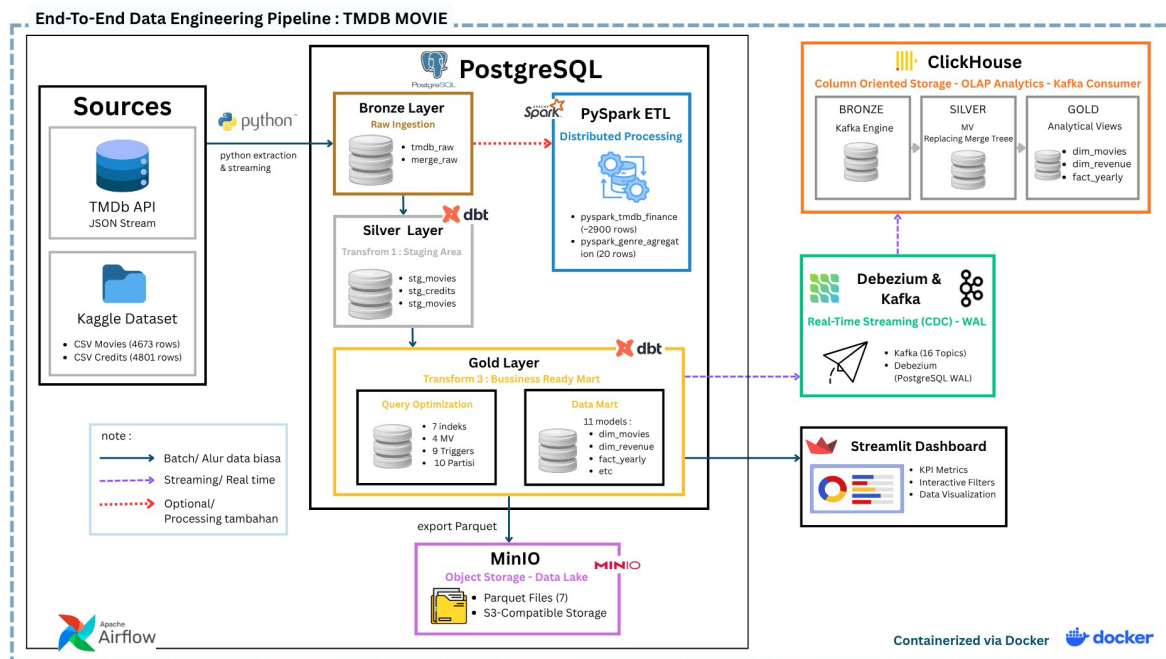
- TMDb Live API — <https://www.themoviedb.org/subscribe/developer>
- Kaggle Dataset — <https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata>

Pipeline Summary

Step	Component	Output
Step 1	Bronze Layer — Ingestion	tmdb_raw (80 rows), merge_raw (4,803 rows)
Step 2	Silver/Gold — dbt Transformation	11 mart models in schema silver_mart
Step 3	Query Optimization	4 MVs, 10 partitions, 7 indexes, 9 triggers
Step 4	PySpark ETL	pyspark_tmdb_finance, pyspark_genre_aggregation
Step 5	Apache NiFi Visual Pipeline	nifi_tmdb_movie
Step 6	Kafka + Debezium CDC	16 topics, real-time streaming
Step 7	ClickHouse OLAP	9 objects, 9,604 rows dim_movies
Step 8	MinIO Object Storage	7 Parquet files, 2.7 MiB
Step 9	Streamlit Dashboard	Interactive dashboard with filters
Step 10	Apache Airflow / DAG Orchestration	Automation of 4 steps (API, CSV to PostgreSQL and MinIO)

Pipeline Architecture

The pipeline uses a modern EtLT pattern (Extract → Load → Transform → Load → Transform), where data is first collected in raw form and then incrementally transformed following Medallion Architecture.



All Docker containers involved must be running before starting this project.

	Name	Container ID	Image	Port(s)	CPU	Actions
☐	tmdb-pipeline-project	-	-	-	N	
☐	tmdb_airflow_scheduler	33c49eaa546f	apache/airflow:2.8.1		N	
☐	tmdb_airflow_web	457fb22a156e	apache/airflow:2.8.1	8082:8080	N	
☐	tmdb_airflow_init	de3683fc0b9a	apache/airflow:2.8.1		N	
☐	tmdb_debezium	a8c8d14f0c42	debezium/connect:2.4	8083:8083	N	
☐	tmdb_pgadmin	c0e92ce2849b	dpape/pgadmin4	5050:80	N	
☐	tmdb_kafka	17fcf1c8b125	confluentinc/cp-kafka:7.4.0	29092:29092	N	
☐	tmdb_kafka_man	8a0a6fc83902	hlebalbau/kafka-manager:latest	9000:9000	N	
☐	tmdb_postgres	22827ae4eea3	postgres:13	5432:5432	N	
☐	tmdb_clickhouse	4ce4f4eac5e7	clickhouse/clickhouse-server:23.8	18123:8123 Show all ports (2)	N	
☐	tmdb_zookeeper	7dda08543494	confluentinc/cp-zookeeper:7.4.0		N	
☐	tmdb_minio	4a23ad4f533a	minio/minio:latest	9002:9000 Show all ports (2)	N	

Step 1 — Bronze Layer: Environment Setup & Data Ingestion

The Bronze layer is the first layer of Medallion Architecture, storing raw data without any transformation. Data comes from two sources that are then merged.

1.1 Docker Infrastructure (Containers)

Container	Image	Port	Function
tmdb_postgres	postgres:13	5432:5432	Main PostgreSQL database

tmdb_pgadmin	dpage/pgadmin4	5050:80	Database monitoring GUI
--------------	----------------	---------	-------------------------

pgAdmin connection config: Host = postgres (container name, NOT localhost) | Port = 5432
 | Username = admin | Password = adminadmin | Database = tmdb_db

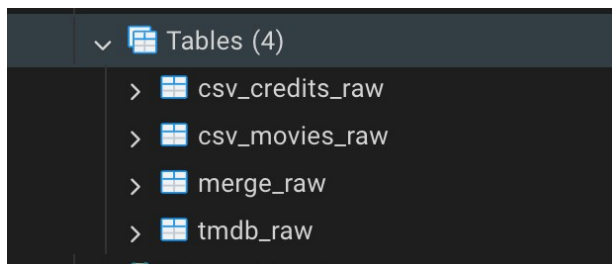
1.2 Data Sources

Source	Format	Row Count	Notes
TMDb Live API (tmdb_raw)	JSON	80 rows	/movie/popular endpoint, authenticated via API key
Kaggle CSV — tmdb_5000_movies	CSV	4,673 rows	Historical film data with budget, revenue, genres columns
Kaggle CSV — tmdb_5000_credits	CSV	4,801 rows	Cast and crew data per film

1.3 Bronze Structure in pgAdmin

Navigate to <http://localhost:5050/browser/>

Project name: tmdb_postgres → tmdb_db → Schemas → public → Tables → merge_raw, tmdb_raw, csv_movies_raw



1.4 tmdb_raw Table Structure (15 Columns) from TMDb API

Column	Data Type	Description
id	bigint	Unique film ID from TMDb
title	text	Film title in English
original_title	text	Original film title
original_language	text	Language code (en, fr, ko, etc.)
overview	text	Film description / synopsis
genre_ids	text	List of genre IDs (still in JSON string format)
release_date	text	Release date (format: YYYY-MM-DD)
popularity	double precision	Popularity score from TMDb
vote_average	double precision	Average rating (scale 0–10)
vote_count	bigint	Total number of votes

adult	boolean	Adult content (True/False)
video	boolean	Whether it is a video (usually False)
backdrop_path	text	Backdrop image path
poster_path	text	Film poster path
category	text	Category (popular/top_rated/etc.)

1.5 merge_raw Table Structure (22 Columns, 4,803 rows)

Two CSV files are merged into the merge_raw table with shape (4803, 22). Available columns:

budget, genres, homepage, id, keywords, original_language, original_title, overview, popularity, production_companies, production_countries, release_date, revenue, runtime, spoken_languages, status, tagline, title, vote_average, vote_count, cast, crew

1.6 JSON Columns to be Parsed

NOTE: The following columns are still in JSON STRING format and will be parsed in subsequent steps.

JSON Column	Example Content	Parsed in Step
genres	[{"id":28,"name":"Action"},...]	Step 2 (dbt) & Step 4 (PySpark)
cast	[{"name":"Sam Worthington"},...]	Step 2 (dbt) & Step 4 (PySpark)
crew	[{"job":"Director","name":"..."}]	Step 2 (dbt) & Step 4 (PySpark)
keywords	[{"id":1,"name":"avatar"},...]	Step 2 (dbt)
production_companies	[{"name":"Warner Bros"},...]	Step 2 (dbt) & Step 5 (NiFi)
production_countries	[{"iso":"US","name":"..."},...]	Step 2 (dbt)
spoken_languages	[{"iso":"en","name":"English"},...]	Step 2 (dbt)

The screenshot shows the DBeaver database client interface. On the left, the 'Object Explorer' pane displays the 'Columns (22)' for the 'merge_raw' table. The columns are: budget (bigint), genres (text), homepage (text), id (bigint), keywords (text), original_language (text), original_title (text), overview (text), popularity (double precision), production_companies (text), production_countries (text), release_date (text), revenue (bigint), runtime (double precision), spoken_languages (text), status (text), tagline (text), title (text), vote_average (double precision), vote_count (bigint), cast (text), and crew (text). The main pane shows a SQL query: 'SELECT * FROM public.merge_raw LIMIT 8;'. The 'Data Output' pane displays the first 8 rows of the query result, showing columns: budget, genres, homepage, and others. The status bar at the bottom indicates 'Total rows: 8' and 'Query complete 00:00:00.164'.

id	budget	genres	homepage
1	237000000	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}, {"id": 878, "name": "Science Fiction"}]	http://www.avatarmov
2	300000000	[{"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}, {"id": 28, "name": "Action"}]	http://disney.go.com/e
3	245000000	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 80, "name": "Crime"}]	http://www.sony pictur
4	250000000	[{"id": 28, "name": "Action"}, {"id": 80, "name": "Crime"}, {"id": 18, "name": "Drama"}, {"id": 53, "name": "Thriller"}]	http://www.thedarkkn
5	260000000	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 878, "name": "Science Fiction"}]	http://movies.disney.c
6	41000000	[{"id": 35, "name": "Comedy"}]	http://www.iloveyoum
7	258000000	[{"id": 14, "name": "Fantasy"}, {"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}]	http://www.sony pictur
8	260000000	[{"id": 16, "name": "Animation"}, {"id": 10751, "name": "Family"}]	http://disney.go.com/e

Step 2 — Silver & Gold Layer: dbt Transformation

dbt (data build tool) is used to transform data from the Bronze layer into clean, structured analytic tables. The architecture uses 2 layers: Staging (cleansing) and Mart (analytics).

Location in pgAdmin: tmdb_db → Schemas → silver_mart_silver_mart → 11 Tables (dim_movies, dim_revenue, dim_genres, etc.) → Views (stg_movies, stg_credits, stg_tmdbapi)

2.1 dbt Environment

Component	Value
dbt version	1.11.7 (latest)
Python version	3.11.9
PostgreSQL adapter	dbt-postgres 1.10.0 — OK
Target schema	silver_mart
Total models	3 staging + 11 mart = 14 models total

2.2 Staging Layer (3 Models)

Model	Source	Key Transformations
stg_movies.sql	merge_raw	Handle 2 date formats (DD/MM/YYYY & YYYY-MM-DD), parse all JSON columns, normalize popularity & vote_average
stg_credits.sql	merge_raw	Extract lead actor, top 3 cast, director, screenplay writer, producer, cinematographer, music composer from JSON
stg_tmdbapi.sql	tmdb_raw	Fix date format, handle genre_ids as a native PostgreSQL array

2.3 Mart Layer (11 Models) — Schema: silver_mart

Model	Description	Key Columns
dim_movies	Complete master film table, merged from 3 sources	movie_id, title, primary_genre, director, lead_actor, release_year
dim_genres	Performance summary per genre	genre, total_films, total_revenue, avg_rating, top_grossing_film
dim_directors	Director career profile	director, total_films, director_roi_pct, filmography
dim_actor	Actor career profile (lead + supporting)	actor_name, lead_rate_pct, frequent_director, best_performing_film

dim_country	Summary per production country	primary_country, total_films, most_common_genre, most_active_director
dim_language	Film analysis per language	original_language, total_films, dominant_country, top_revenue_genre
dim_company	Production house profile	main_production_company, blockbuster_rate_pct, most_active_director
dim_revenue	Financial facts per film	performance_label, roi_pct, popularity_tier, profit
fact_vs	Reconciliation of 2 data sources (CSV vs API)	in_api, in_kaggle, popularity_diff, api_vote_average
fact_yearly	Industry trends per year per genre	release_year, primary_genre, blockbuster_count, loss_count
dim_yearlysum	Industry summary per year (1 row/year)	most_active_director, top_grossing_film, most_common_genre

2.4 Film Performance Categories (from mart layer dim_revenue)

Label	Criteria	Description
Mega Blockbuster	revenue $\geq$ budget $\times$ 3	Highly successful film, 3 $\times$ return on investment
Blockbuster	revenue $\geq$ budget $\times$ 2	Successful film, 2 $\times$ return on investment
Profitable	revenue > budget	Profitable but not significantly
Break Even	revenue = budget	Neither profit nor loss
Loss	revenue < budget	Film at a loss

	table_schema name	table_name name
1	public	merge_raw
2	public	tmdb_raw
3	silver_mart	dim_actor
4	silver_mart	dim_company
5	silver_mart	dim_country
6	silver_mart	dim_directors
7	silver_mart	dim_genres
8	silver_mart	dim_language
9	silver_mart	dim_movies
10	silver_mart	dim_revenue
11	silver_mart	dim_yearlysu...
12	silver_mart	fact_vs
13	silver_mart	fact_yearly

Step 3 — Query Optimization: EXPLAIN, Index, Partition, MV

Step 3 demonstrates database performance optimization capability using 4 main techniques: EXPLAIN for profiling, Indexes for faster lookups, Partitions for splitting large data, and Materialized Views as aggregation caches.

Location in pgAdmin: tmdb_db → Schemas → silver_mart_silver_mart → Tables → dim_movies → right-click → Properties → Indexes (view 7 indexes there)

3.1 Final Validation Results

Object	Count	Details
Indexes	7	idx_dim_movies_director, idx_dim_movies_genre, idx_dim_movies_year, idx_dim_movies_genre_rating, + 3 others
Materialized Views	4	mv_genre_summary, mv_revenue_summary, mv_director_performance, mv_country_performance
Triggers	9	3 auto-refresh functions × 3 events (INSERT/UPDATE/DELETE) = 9 triggers
Partitions	10	fact_yearly_1920s through fact_yearly_2010s (per decade)

3.2 EXPLAIN — Query Performance Analysis

EXPLAIN was run on dim_movies and fact_yearly as a baseline before optimization. Output showed Seq Scan (reading the entire table), which then changed to Index Scan after indexes were created — cost dropped drastically.

3.3 fact_yearly Partitions — Per Decade

The fact_yearly table is divided into 10 sub-tables based on release_year using PARTITION BY RANGE. When a query filters with WHERE release_year = 2010, PostgreSQL only reads the fact_yearly_2010s partition — not the entire table.

	QUERY PLAN
	text
1	Sort (cost=13.24..13.31 rows=27 width=36)
2	Sort Key: release_year
3	-> HashAggregate (cost=12.26..12.60 rows=27 width=36)
4	Group Key: release_year
5	-> Seq Scan on fact_yearly (cost=0.00..10.45 rows=363 w...
6	Filter: ((release_year >= 1990) AND (release_year <= 20...

Then create a index

```
Query Query History
1 CREATE INDEX idx_dim_movies_director ON silver_mart.dim_movies (director);
2 CREATE INDEX idx_dim_movies_genre ON silver_mart.dim_movies (primary_genre);
3 CREATE INDEX idx_dim_movies_year ON silver_mart.dim_movies (release_year);
4 CREATE INDEX idx_dim_movies_genre_rating ON silver_mart.dim_movies (primary_genre, vote_average DESC);
5 CREATE INDEX idx_dim_revenue_genre ON silver_mart.dim_revenue (primary_genre);
6 CREATE INDEX idx_dim_revenue_year ON silver_mart.dim_revenue (release_year);
7 CREATE INDEX idx_dim_revenue_perf ON silver_mart.dim_revenue (performance_label);

Data Output Messages Notifications
CREATE INDEX
Query returned successfully in 381 msec.
```

```
Query Query History
2 FROM silver_mart.dim_revenue WHERE director = 'Steven Spielberg';
3
4 EXPLAIN SELECT title, vote_average, popularity, revenue
5 FROM silver_mart.dim_revenue
6 WHERE primary_genre = 'Action' AND vote_average > 7.0
7 ORDER BY revenue DESC LIMIT 10;

Data Output Messages Notifications
Showing rows: 1 to 8 Page No:

QUERY PLAN
text
1 Limit (cost=79.52..79.53 rows=1 width=31)
2 -> Sort (cost=79.52..79.53 rows=1 width=31)
3 Sort Key: revenue DESC
4 -> Bitmap Heap Scan on dim_revenue (cost=8.69..79.51 rows=1 width=31)
5 Recheck Cond: (primary_genre = 'Action')::text
6 Filter: (vote_average > 7.0)
7 -> Bitmap Index Scan on idx_dim_revenue_genre (cost=0.00..8.69 rows=588 width=31)
8 Index Cond: (primary_genre = 'Action')::text
```

```
Query Query History
1 -- Tabel LAMA (tanpa partisi)
2 EXPLAIN SELECT * FROM silver_mart.fact_yearly WHERE release_year = 2010;
3
4 -- Tabel BARU (dengan partisi) - bandingkan cost-nya
5 EXPLAIN SELECT * FROM public.fact_yearly_partisi WHERE release_year = 2010;

Data Output Messages Notifications
Showing rows: 1 to 2 Page No: 1

QUERY PLAN
text
1 Seq Scan on fact_yearly_2010s fact_yearly_partisi (cost=0.00..3.28 rows=15 width=31)
2 Filter: (release_year = 2010)
```

3.4 Materialized View + Auto-Refresh Trigger

4 Materialized Views were created as aggregation caches for the dashboard. Every data change in the source table automatically triggers an MV REFRESH without manual intervention.

Materialized View	Source	Insights Provided
mv_genre_summary	dim_genres	Most profitable genre, most popular, annual trends

mv_revenue_summary	dim_revenue	Booming genres per decade, film industry trends
mv_director_performance	dim_directors	Most profitable directors, career duration
mv_country_performance	dim_country	Dominant countries per genre, favorite genre per country

Query Query History	
1 SELECT * FROM public.mv_country_performance	
Data Output Messages Notifications	
Showing rows: 1 to 28 Page No: 1 of 1	
	primary_country text
1	United States of Ameri...
2	United Kingdom
3	Germany

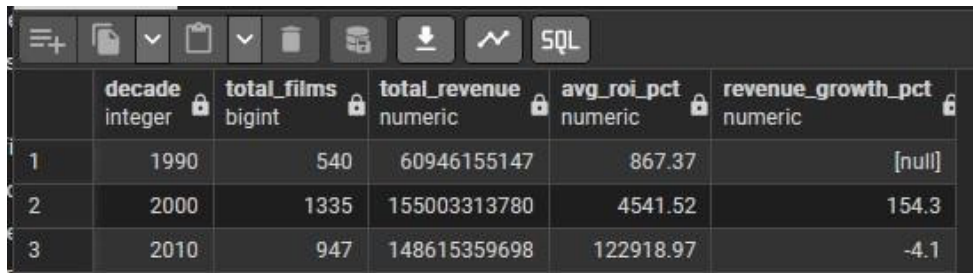
Query Query History	
1 SELECT * FROM public.mv_revenue_summary	
Data Output Messages Notifications	
Showing rows: 1 to 119 Page No: 1 of 1	
	primary_genre text
1	Action
2	Adventure
3	Action
4	Comedy
5	Drama
6	Adventure
7	Comedy

Query Query History	
1 SELECT * FROM public.mv_director_performance	
Data Output Messages Notifications	
Showing rows: 1 to 509 Page No: 1 of 1	
	director text
1	David Hand
2	Alex Kendrick
3	Victor Fleming
4	Darren Lynn Bousman
5	Guy Hamilton
6	Henry Joost

Query Query History	
1 SELECT * FROM public.mv_genre_summary	
Data Output Messages Notifications	
Showing rows: 1 to 19 Page No: 1 of 1	
	genre text
1	Action
2	Adventure
3	Drama
4	Comedy
5	Animation
6	Fantasy
7	Science Ficti...

3.5 Analytic CTEs

CTEs (Common Table Expressions) were used to make queries more readable, with 3 ready-to-use analytic insights: top films per genre, most consistent directors, and industry trends per decade.



	decade integer	total_films bigint	total_revenue numeric	avg_roi_pct numeric	revenue_growth_pct numeric
1	1990	540	60946155147	867.37	[null]
2	2000	1335	155003313780	4541.52	154.3
3	2010	947	148615359698	122918.97	-4.1

Step 4 — PySpark ETL: Distributed Data Processing

PySpark is used to process merge_raw data using a distributed in-memory processing approach — unlike pandas which can only run on a single machine. This demonstrates the capability to handle data at scales pandas cannot.

4.1 Comparison: pandas vs PySpark

Aspect	pandas	PySpark
Execution	Eager — processed immediately	Lazy — only executes on .show()/.write()
Data limit	Max RAM of laptop (~8–32 GB)	Petabytes (cluster scale)
SQL support	Not direct	spark.sql() — full SQL
Best for	Exploration, prototyping	Production pipelines, big data

4.2 PySpark ETL Flow

EXTRACT → merge_raw from PostgreSQL via JDBC (4,803 rows, 22 columns)

TRANSFORM 1 — JSON column extraction:

- genres → primary_genre
- production_companies → primary_company
- spoken_languages → primary_language

TRANSFORM 2 — New column calculations:

- profit = revenue - budget
- roi_pct = (revenue - budget) / budget × 100
- performance_label: Mega Blockbuster / Blockbuster / Profitable / Break Even / Loss
- popularity_tier: Viral (>100) / Popular (>50) / Known (>20) / Niche

TRANSFORM 3 — Aggregation per genre (groupBy primary_genre):

- total_films, avg_rating, total_revenue, total_profit, avg_roi_pct, mega_blockbuster_count, loss_count

LOAD 1 → public.pyspark_tmdb_finance (per-film data, complete) LOAD 2 → public.pyspark_genre_aggregation (per-genre aggregation, 20 rows)

4.3 Validation Results

Table	Row Count	Notes
pyspark_tmdb_finance	~2,900	Films with budget > 0 AND revenue > 0
pyspark_genre_aggregation	20	1 row per genre (Action, Drama, Comedy, etc.)

The screenshot shows a database interface with a sidebar on the left listing tables like 'fact_yearly_partisi', 'merge_raw', 'pyspark_genre_aggregation', 'pyspark_tmdb_finance', 'pyspark_tmdb_result', 'tmdb_raw', 'Trigger Functions', 'Types', and 'Views'. The main area displays a query: `SELECT COUNT(*) FROM public.pyspark_tmdb_finance;`. Below the query, the 'Data Output' tab shows a single row with the count 4803.

	count
1	4803

The screenshot shows a database interface with a sidebar on the left listing tables like 'fact_yearly_partisi', 'merge_raw', 'pyspark_genre_aggregation', 'pyspark_tmdb_finance', 'pyspark_tmdb_result', 'tmdb_raw', 'Trigger Functions', 'Types', and 'Views'. The main area displays a query: `SELECT performance_label, COUNT(*) as total FROM public.pyspark_tmdb_finance GROUP BY performance_label ORDER BY total DESC;`. Below the query, the 'Data Output' tab shows a table with 5 rows.

	performance_label	total
1	Mega Blockbuster	2313
2	Loss	1326
3	Profitable	629
4	Blockbuster	533
5	Break Even	2

The screenshot shows a database interface with a sidebar on the left listing tables like 'fact_yearly_partisi', 'merge_raw', 'pyspark_genre_aggregation', 'pyspark_tmdb_finance', 'pyspark_tmdb_result', 'tmdb_raw', 'Trigger Functions', 'Types', and 'Views'. The main area displays a query: `SELECT primary_genre, total_films, total_revenue, avg_roi_pct FROM public.pyspark_genre_aggregation ORDER BY total_revenue DESC;`. Below the query, the 'Data Output' tab shows a table with 7 rows.

	primary_genre	total_films	total_revenue	avg_roi_pct
1	Action	588	91459088189	187.51
2	Adventure	288	70872316207	471.22
3	Drama	747	54101757502	1138398.15
4	Comedy	634	53037118804	425.95
5	Animation	99	29595221289	620.67
6	Fantasy	93	17057198167	370.35
7	Science Fiction	79	16176309703	418.92

Step 6 — Apache Kafka + Debezium CDC: Real-Time Streaming

Step 6 builds a Change Data Capture (CDC) system that captures every data change in PostgreSQL in real time and streams it to Kafka. This is an architectural pattern used at massive scale, such as Uber (130 million events/second).

Access the Kafka Cluster at: <http://localhost:9000>

6.1 Kafka Stack Components

Container	Image	Port	Function
tmdb_zookeeper	confluentinc/cp-zookeeper:7.4.0	2181	Kafka cluster coordination
tmdb_kafka	confluentinc/cp-kafka:7.4.0	29092:29092	Message broker, event stream storage
tmdb_debezium	debezium/connect:2.4	8083:8083	CDC connector PostgreSQL → Kafka
tmdb_kafka_man	hlebalbau/kafkamanager	9000:9000	Kafka web UI monitoring

6.2 PostgreSQL WAL Configuration

For Debezium to capture changes, PostgreSQL must be set to WAL logical mode:

```
# Enter the PostgreSQL container
docker exec -it tmdb_postgres bash
```

```
# Set WAL level to logical
echo "wal_level = logical" >> /var/lib/postgresql/data/postgresql.conf
```

```
# Restart container
docker restart tmdb_postgres
-- Verify in pgAdmin:
SHOW wal_level; -- must show: logical
```

```
-- Check Debezium publication:
SELECT * FROM pg_publication;
-- pubname: dbz_publication | puballtables: true
-- pubinsert/pubupdate/pubdelete: true
```

6.3 Result: 16 Kafka Topics

Debezium automatically creates 1 topic per captured table:

Kafka Topic	Source Table
tmdb.silver_mart.dim_actor	silver_mart.dim_actor
tmdb.silver_mart.dim_company	silver_mart.dim_company
tmdb.silver_mart.dim_country	silver_mart.dim_country
tmdb.silver_mart.dim_directors	silver_mart.dim_directors
tmdb.silver_mart.dim_genres	silver_mart.dim_genres
tmdb.silver_mart.dim_language	silver_mart.dim_language
tmdb.silver_mart.dim_movies	silver_mart.dim_movies
tmdb.silver_mart.dim_revenue	silver_mart.dim_revenue
tmdb.silver_mart.dim_yearlysum	silver_mart.dim_yearlysum

tmdb.silver_mart.fact_vs	silver_mart.fact_vs
tmdb.silver_mart.fact_yearly	silver_mart.fact_yearly
+ 5 internal Kafka topics	__consumer_offsets, connect_configs, etc.

Topic	# Partitions	# Brokers	Brokers Spread %	Brokers Skew %	Brokers Leader Skew %	# Replicas	Under Replicated %
__consumer_offsets	50	1	100	0	0	1	0
connect_configs	1	1	100	0	0	1	0
connect_offsets	25	1	100	0	0	1	0
connect_statuses	5	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_actors	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_company	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_country	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_directors	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_genres	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_language	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_movies	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_revenue	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_yearlysum	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.fact_vs	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.fact_yearly	1	1	100	0	0	1	0

Step 7 — ClickHouse OLAP: Column-Oriented Analytics

ClickHouse is a column-oriented OLAP database that is significantly faster for aggregation analytics compared to PostgreSQL (row-oriented). This is where Medallion Architecture is reimplemented within ClickHouse: Bronze (Kafka Engine) → Silver (ReplacingMergeTree) → Gold (View).

Access ClickHouse at: <http://localhost:18123/play>

7.1 PostgreSQL vs ClickHouse Differences

Aspect	PostgreSQL (OLTP)	ClickHouse (OLAP)
Storage method	Per row (row-oriented)	Per column (column-oriented)
Analytic speed	Slower for aggregations	Much faster for column analysis
Use case	Transactions: orders, banking	Analytics: BI, Spotify Wrapped
MV auto-update	No — requires manual trigger	Yes — automatic without triggers

7.2 3-Layer Architecture in ClickHouse

BRONZE: Kafka Engine Table (dim_movies_queue) — Directly consumes from Kafka topic via ENGINE = Kafka.

SILVER: ReplacingMergeTree Table (dim_movies) — Stores final data, prevents duplicates via ENGINE = ReplacingMergeTree(insert_time).

GOLD: View (tmdb_movies_gold) — Adds calculated columns: profit, performance_label, popularity_tier. WHERE budget > 0 AND revenue > 0.

7.3 Objects in ClickHouse (9 Total Objects)

Object Name	Type	Engine / Notes
dim_movies	Base Table	ReplacingMergeTree — final silver layer data
dim_movies_queue	Kafka Queue	Kafka Engine — consumes from topic
dim_movies_mv	Materialized View	Bridge: queue → base table (automatic)
dim_revenue	Base Table	ReplacingMergeTree
dim_revenue_queue	Kafka Queue	Kafka Engine
dim_revenue_mv	Materialized View	Automatic
fact_yearly	Base Table	ReplacingMergeTree
fact_yearly_queue	Kafka Queue	Kafka Engine
fact_yearly_mv	Materialized View	Automatic

7.4 Data Validation Results in ClickHouse

Table	Row Count	Notes
dim_movies	4,802 rows	Data flowing from Kafka CDC
dim_revenue	3,229 rows	Films with complete financial data
fact_yearly	363 rows	Aggregation per year per genre



Step 8 — MinIO Object Storage: Modern HDFS Replacement

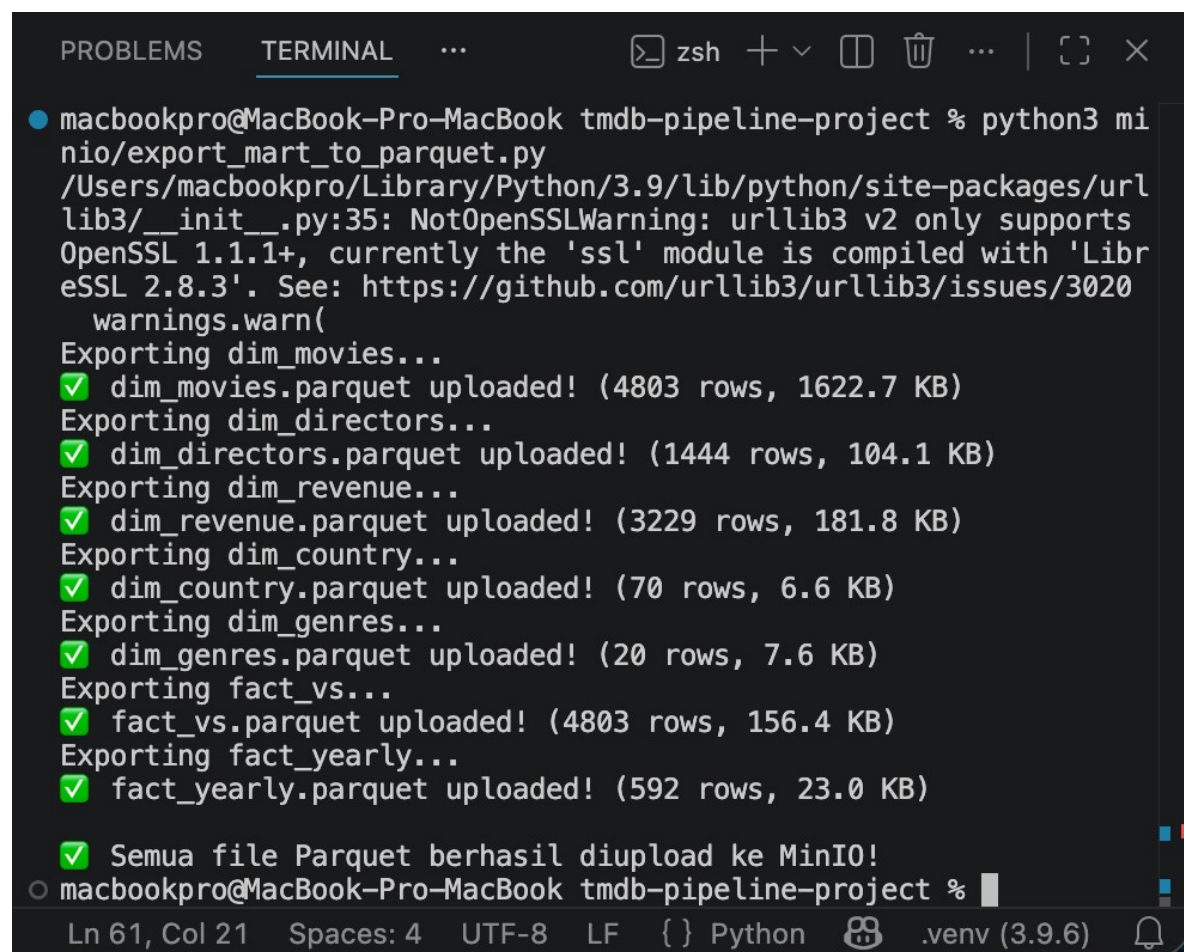
MinIO is a modern object storage system compatible with Amazon S3, used to store mart layer results in Parquet format. Login: minioadmin / minioadmin

Access MinIO at: <http://localhost:9001/browser>

8.1 Configuration

```
docker run -d -p 9002:9000 -p 9001:9001 --name tmdb_minio \
-v C:\data\tmdb-minio:/data \
-e MINIO_ROOT_USER=minioadmin \
-e MINIO_ROOT_PASSWORD=minioadmin \
quay.io/minio/minio server /data --console-address ':9001'
```

Web UI: <http://localhost:9001> | Username: minioadmin | Password: minioadmin



```
PROBLEMS  TERMINAL  ...  zsh  +  -  [ ]  [ ]  ...  |  [ ]  x

● macbookpro@MacBook-Pro-MacBook tmdb-pipeline-project % python3 minio/export_mart_to_parquet.py
/Users/macbookpro/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
Exporting dim_movies...
✓ dim_movies.parquet uploaded! (4803 rows, 1622.7 KB)
Exporting dim_directors...
✓ dim_directors.parquet uploaded! (1444 rows, 104.1 KB)
Exporting dim_revenue...
✓ dim_revenue.parquet uploaded! (3229 rows, 181.8 KB)
Exporting dim_country...
✓ dim_country.parquet uploaded! (70 rows, 6.6 KB)
Exporting dim_genres...
✓ dim_genres.parquet uploaded! (20 rows, 7.6 KB)
Exporting fact_vs...
✓ fact_vs.parquet uploaded! (4803 rows, 156.4 KB)
Exporting fact_yearly...
✓ fact_yearly.parquet uploaded! (592 rows, 23.0 KB)

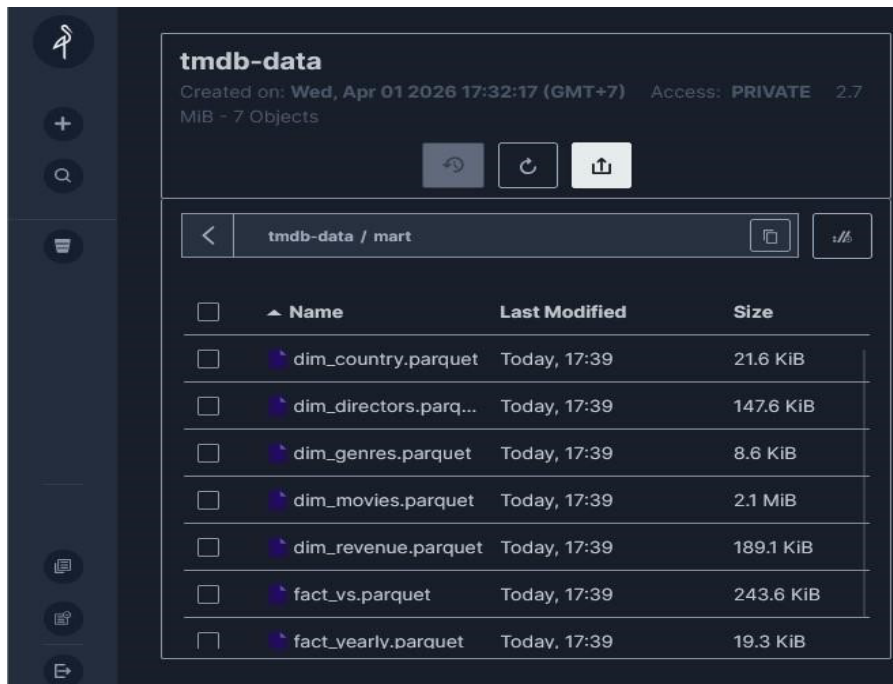
✓ Semua file Parquet berhasil diupload ke MinIO!
○ macbookpro@MacBook-Pro-MacBook tmdb-pipeline-project %
```

Ln 61, Col 21 Spaces: 4 UTF-8 LF { } Python .venv (3.9.6)

8.2 Upload Results — Bucket: tmdb-data/mart/

Total: 7 Parquet files, 2.7 MiB

Parquet File	Row Count	Size	Upload Date
dim_movies.parquet	4,802	2.1 MiB	01 Apr 2026, 17:39
dim_directors.parquet	2,349	147.6 KiB	01 Apr 2026, 17:39
fact_vs.parquet	4,867	243.6 KiB	01 Apr 2026, 17:39
dim_revenue.parquet	3,229	189.1 KiB	01 Apr 2026, 17:39
dim_country.parquet	70	21.6 KiB	01 Apr 2026, 17:39
fact_yearly.parquet	363	19.3 KiB	01 Apr 2026, 17:39
dim_genres.parquet	19	8.6 KiB	01 Apr 2026, 17:39



Step 9 — Streamlit Dashboard: Data Visualization

An interactive dashboard built with Streamlit (Python library), connected directly to data in PostgreSQL. The dashboard displays KPI scorecards, charts, and tables with interactive filters.

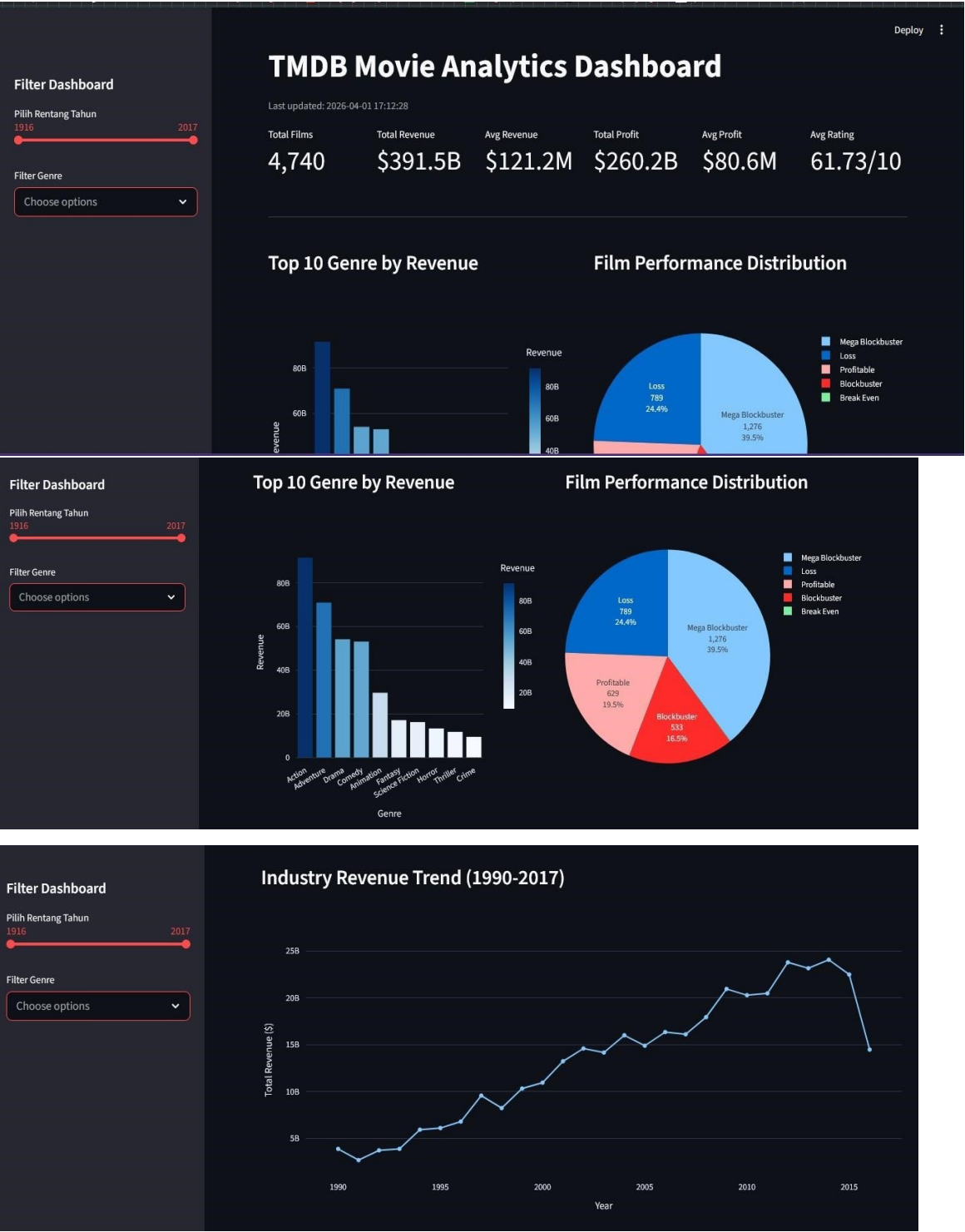
Access the Streamlit dashboard at: <http://localhost:8501/>

9.1 Dashboard Features

Feature	Description
Year Range Filter	Slider to select release year range (e.g., 1959–2004)
Genre Filter	Multi-select to filter by genre (Comedy, Adventure, etc.)
KPI Scorecards	Total Films, Total Revenue, Avg Revenue, Total Profit, Avg Profit, Avg Rating

Top 10 Genre by Revenue	Bar chart of revenue per genre
Film Performance Distribution	Pie chart of Mega Blockbuster / Blockbuster / Profitable / Loss distribution
Top 20 Highest Revenue Films	Detail table with Title, Genre, Lead Actor, Director, Revenue, Performance columns

9.2 Dashboard Sample Data



Filter Dashboard

Pilih Rentang Tahun
19162017

Filter Genre
Choose options

Top 10 Genre by Revenue

Film Performance Distribution

Filter Dashboard

Pilih Rentang Tahun
19162017

Filter Genre
Choose options

Industry Revenue Trend (1990-2017)



Filter: 1959–2004, Genre: Comedy + Adventure Highest-grossing film in this filter: The Lord of the Rings: The Return of the King (\$1.118B revenue, Adventure, director Peter Jackson)



Step 10 — Airflow DAG Orchestration

Apache Airflow is used to automate and schedule the end-to-end TMDB pipeline. The DAG (Directed Acyclic Graph) named `tmdb_pipeline_dag` manages the sequential execution of 5 tasks on an automated schedule.

Access Airflow UI: <http://localhost:8082>

10.1 DAG Configuration

Component	Value
DAG ID	tmdb_pipeline_dag
Description	TMDB End-to-End Pipeline
Schedule	0 0 * * 1 (every Monday at midnight)
Next Run	2026-06-01, 00:00:00
Catchup	False
Owner	nabila
Retries per Task	1 (retry delay: 5 minutes)

10.2 Task Sequence in the DAG

Order	Task ID	Function
1	fetch_tmdb_api	Fetch latest film data from TMDb live API → save to tmdb_raw
2	load_kaggle_csv	Load Kaggle CSV files into PostgreSQL (csv_movies_raw, csv_credits_raw)
3	merge_raw	Merge both sources into the merge_raw table (4,803 rows)
4	export_to_minio	Export 7 mart tables from PostgreSQL → MinIO in Parquet format
5	export_to_bigquery	Read 7 Parquet files from MinIO → Upload to Google BigQuery (tmdb_gold)

Execution flow: fetch_tmdb_api → load_kaggle_csv → merge_raw → export_to_minio → export_to_bigquery

Execution is sequential — if one task fails, subsequent tasks are not run.

10.3 DAG Execution Results

Metric	Value
Last Run Status	✅ Success (all 5 tasks green)
Successful Run Timestamp	2026-06-02, 03:40:04 UTC
Maximum Duration	00:05:53

Results per Task (Last Run):

Task	Status	Output
------	--------	--------

fetch_tmdb_api	✅ Success	80 rows → tmdb_raw
load_kaggle_csv	✅ Success	4,673 + 4,801 rows → PostgreSQL
merge_raw	✅ Success	4,803 rows → merge_raw
export_to_minio	✅ Success	7 Parquet files → tmdb-data/mart/
export_to_bigquery	✅ Success	7 tables → BigQuery dataset tmdb_gold

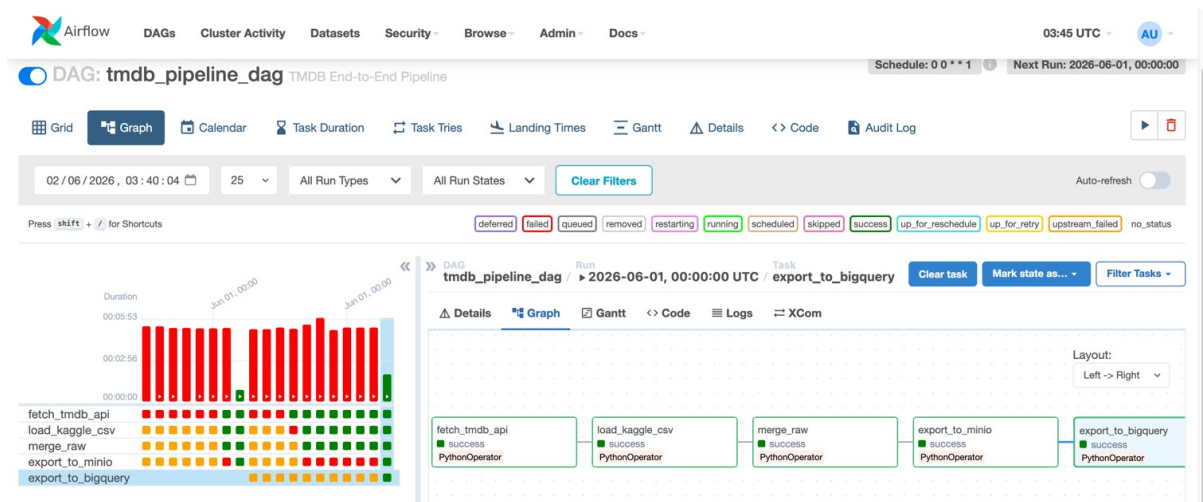
Development Notes: Previous run failures occurred during the environment configuration phase. Root causes included connecting to PostgreSQL using localhost instead of the container name tmdb_postgres; the MinIO endpoint using a container name with an underscore incompatible with boto3 (resolved by using IP address 172.18.0.2:9000 + Config(signature_version='s3v4')); and the dbt schema needing to be adjusted from silver_mart to silver_mart_silver_mart. The final run completed successfully after all configurations were corrected.

10.4 What Airflow Controls vs What Runs Independently

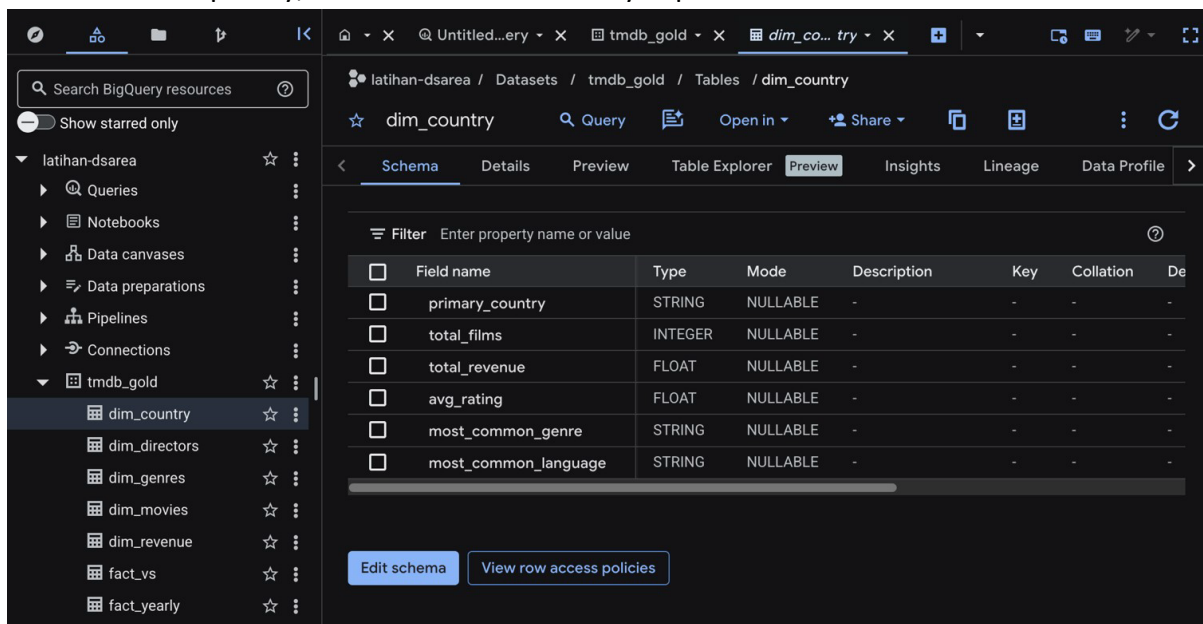
Component	Notes
✅ Controlled by DAG	fetch_tmdb_api → load_kaggle_csv → merge_raw → export_to_minio → export_to_bigquery
⚡ Runs Independently	Kafka + Debezium (real-time CDC), ClickHouse (auto-consume), Streamlit (always on)

The screenshot shows the Google Cloud IAM & Admin console for a service account named 'tmdb-pipeline'. The breadcrumb navigation is 'IAM & Admin / Service accounts / Service account: 106637632176920980692'. The 'tmdb-pipeline' service account is selected, and the 'Details' tab is active. The 'Service account details' section shows the name 'tmdb-pipeline' and description 'Service account untuk TMDB pipeline ke BigQuery'. The email address is 'tmdb-pipeline@latihan-dsarea.iam.gserviceaccount.com' and the unique ID is '106637632176920980692'. The 'Service account status' section shows the account is 'Enabled' with a green checkmark and a 'Disable service account' button.

On Airflow screen = 5 task succeeded



The result in BiqQuery, 7 file from MiniO already expored.



Appendix: Summary of All Database Objects

1) PostgreSQL — tmdb_db

Schema	Table / Object	Type	Notes
public	merge_raw	Table	Bronze: 4,803 rows, 22 columns, result of merging 2 CSVs
public	tmdb_raw	Table	Bronze: 80 rows from TMDb API
public	fact_yearly_partisi	Parent Table	RANGE partition based on release_year

public	fact_yearly_1920s ... 2010s	Partitions (10×)	Sub-table per decade
public	pyspark_tmdb_finance	Table	PySpark ETL result — per film with calculations
public	pyspark_genre_aggregation	Table	PySpark ETL result — per-genre aggregation (20 rows)
public	pyspark_tmdb_result	Table	Initial PySpark ETL result (simplified version)
public	nifi_tmdb_movie	Table	Apache NiFi pipeline result from CSV
silver_mart	dim_movies	dbt Table	Complete master film — 4,802 rows
silver_mart	dim_genres	dbt Table	Per-genre summary — 19 rows
silver_mart	dim_directors	dbt Table	Director profiles — 2,349 rows
silver_mart	dim_actor	dbt Table	Actor profiles
silver_mart	dim_country	dbt Table	Per-country summary — 70 rows
silver_mart	dim_language	dbt Table	Per-language summary
silver_mart	dim_company	dbt Table	Production house profiles
silver_mart	dim_revenue	dbt Table	Financial facts — 3,229 rows
silver_mart	dim_yearlysum	dbt Table	Annual summary
silver_mart	fact_vs	dbt Table	2-source reconciliation — 4,867 rows
silver_mart	fact_yearly	dbt Table	Trends per year per genre — 363 rows
public	mv_genre_summary	MV	Genre aggregation cache
public	mv_revenue_summary	MV	Financial analysis cache per decade
public	mv_director_performance	MV	Director profile cache
public	mv_country_performance	MV	Country profile cache

2) ClickHouse — 9 Objects

Name	Type	Engine	Rows
dim_movies	Base Table	ReplacingMergeTree	9,604
dim_movies_queue	Kafka Table	Kafka Engine	—
dim_movies_mv	Materialized View	—	—
dim_revenue	Base Table	ReplacingMergeTree	6,458
dim_revenue_queue	Kafka Table	Kafka Engine	—
dim_revenue_mv	Materialized View	—	—

fact_yearly	Base Table	ReplacingMergeTree	726
fact_yearly_queue	Kafka Table	Kafka Engine	—
fact_yearly_mv	Materialized View	—	—

```
SHOW TABLES

Query id: 0c75cca0-9a87-4b07-bcf1-d782f275b885

  name
1.  dim_movies
2.  dim_movies_mv
3.  dim_movies_queue
4.  dim_revenue
5.  dim_revenue_mv
6.  dim_revenue_queue
7.  fact_yearly
8.  fact_yearly_mv
9.  fact_yearly_queue

9 rows in set. Elapsed: 0.021 sec.
```

3) Running Docker Containers

Service	Container Name	Image	Port
Airflow Scheduler	tmdb_airflow_scheduler	apache/airflow:2.8.1	—
Airflow Webserver	tmdb_airflow_web	apache/airflow:2.8.1	8082:8080
Airflow Init	tmdb_airflow_init	apache/airflow:2.8.1	—
Debezium Connect	tmdb_debezium	debezium/connect:2.4	8083:8083
pgAdmin	tmdb_pgadmin	dpage/pgadmin4	5050:80
Kafka Broker	tmdb_kafka	confluentinc/cp-kafka:7.4.0	29092:29092
Kafka Manager	tmdb_kafka_man	hlebalbau/kafka-manager:latest	9000:9000
PostgreSQL	tmdb_postgres	postgres:13	5432:5432
ClickHouse	tmdb_clickhouse	clickhouse/clickhouse-server:23.8	18123:8123
ZooKeeper	tmdb_zookeeper	confluentinc/cp-zookeeper:7.4.0	—
MinIO	tmdb_minio	minio/minio:latest	9002:9000